

Real-Time Prediction-Driven Dynamics Simulation to Mitigate Frame Time Variation

Mackinnon Buck
mackinnon.buck@gmail.com
California Polytechnic State University
San Luis Obispo, California, United States

Christian Eckhardt
eckhardt@calpoly.edu
California Polytechnic State University
San Luis Obispo, California, United States

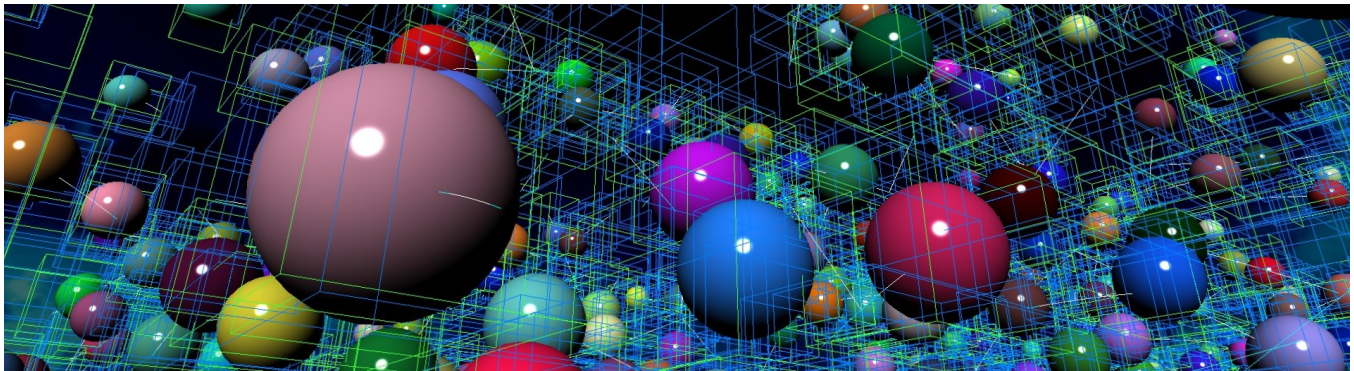


Figure 1: Our work in action. The spheres represent the bodies being simulated, and the wireframe boxes are axis-aligned bounding boxes (AABBs) that encapsulate the bodies' predicted paths of motion.

ABSTRACT

This work introduces a prediction-driven real-time dynamics method that uses a graph-based state buffer to minimize the cost of mispredictions. Our technique reduces the average time needed for dynamics computation on the main thread by running the solver pipeline on a separate thread, enabling interactive multimedia applications to increase the computational budget for graphics at no cost perceptible to the end user.

ACM Reference Format:

Mackinnon Buck and Christian Eckhardt. 2021. Real-Time Prediction-Driven Dynamics Simulation to Mitigate Frame Time Variation. In *SIGGRAPH Asia 2021 Posters (SA '21 Posters)*, December 14–17, 2021, Tokyo, Japan. ACM, New York, NY, USA, 2 pages. <https://doi.org/10.1145/3476124.3488633>

1 INTRODUCTION

Dynamics simulation engines have been applied to numerous industries, including animation, games, and robotics [Grundmann 2009]. Real-time dynamics engines in particular have seen recent performance improvements through techniques like hardware acceleration [Owens et al. 2008] and artificial intelligence [Sanchez-Gonzalez et al. 2020, 2018; Wu et al. 2015]. However, state-of-the-art techniques fail to account for the variation in simulation complexity over time. Sudden increases in interaction complexity between

simulated bodies can momentarily increase the processing time per frame [Shen and Sun 2011]. A naive solution might involve running a simulator in a separate thread, buffering snapshots of the simulation state for future display by the application thread. This approach would mitigate momentary lag spikes, but unpredictable changes to the simulation state would require throwing away all buffered state, rendering the buffer useless. As such, applications like games that rely on continuous, unpredictable human input would not benefit from such an approach.

We present a prediction-driven dynamics method that uses a graph-based state buffer to minimize the cost of mispredictions. Furthermore, our technique reduces the average time needed for dynamics computation on the main thread by separating non-blocking simulation queries from the solver running on a separate thread.

2 OUR APPROACH

Our technique enables human interaction by using a graph-based representation to describe relationships between bodies and groups of bodies over time. An unpredictable change to the simulation only invalidates predicted state affected by the change.

2.1 Sweeps

Rather than stepping the simulation such that all bodies can interact with each other, we simulate each body separately and find overlaps between paths of motion between bodies to identify potential interactions. A sweep is defined by a path with an initial time t_i , a duration t_b , and an axis-aligned bounding box (AABB) capturing the path's physical boundaries, allowing broad-phase collision detection to run only when new sweeps are generated.

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

SA '21 Posters, December 14–17, 2021, Tokyo, Japan

© 2021 Copyright held by the owner/author(s).

ACM ISBN 978-1-4503-8687-6/21/12.

<https://doi.org/10.1145/3476124.3488633>

2.2 The Event Queue

Predicted interactions are solved in order of time of impact (TOI) when possible to minimize the frequency of mispredictions. When a sweep's AABB overlaps another one, we quickly find the estimated time of impact (ETOI) which is the lower bound of the actual time of impact (ATOI). Acceleration structures can be used to organize sweeps hierarchically and increase the performance of sweep overlap detection. Since the ETOI is usually sufficient to determine the order of two interactions, the simulator uses a queue prioritized by ETOI to sort events. When the event with the earliest ETOI is within the user-defined time constant t_b from the visual simulation time t_n , its ATOI is solved. If the ATOI is less than the next event's ETOI, the earliest event has been found.

In addition to tracking interactions between bodies, the event queue contains events linked with the final time of every generated sweep. These events ensure that all bodies have sweeps extending at least t_b time ahead of t_n .

2.3 The Predicted-State Graph

Solved events are added as "outcome" nodes to the predicted-state graph (PSG), which tracks interactions between bodies over time. When an outcome's TOI becomes exceeded by t_n , the outcome node is removed, and the outcome's result sweeps become their respective bodies' root sweeps.

The PSG enables the simulator to determine which branches of the predicted state should be removed given an invalidated body. If a body is invalidated, e.g. from unpredictable user input, all nodes reachable by the invalidated body node are removed, leaving only nodes that were not impacted by the initial change to simulation state. Consequentially, the gaps need to be recomputed to catch back up to $t_b + t_n$. Our approach is to only remove events from the event queue when all involved sweeps get removed from the PSG. When an event is encountered where any involved sweep does not have an outcome, the event is solved, or otherwise skipped. Figure 2 presents an example PSG demonstrating how bodies, sweep nodes, and outcome nodes are related to each other.

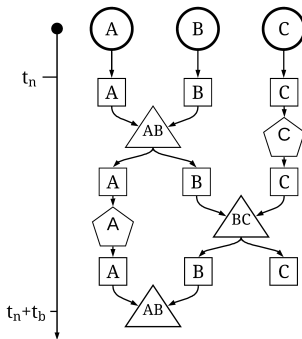


Figure 2: An example predicted-state graph depicting bodies A, B, and C interacting with each other. The squares represent sweep nodes, the triangles interaction outcome nodes, and the pentagons extension outcome nodes. Bodies A and B collide first, followed by B and C, and again A and B.

Table 1: Simulation step times for a scene with varying contact frequencies for 1000 bodies on an Intel Core i7-8700K @3.70GHz.

Contact frequency	Average step time	
	Prediction disabled	Prediction enabled
High	6.516ms	2.510ms
Medium	2.377ms	0.508ms
Low	0.800ms	0.130ms

2.4 Sweep Groups

When bodies cannot be trivially separated from each other, generating their sweeps separately would make the PSG explode with complexity. Thus, rather than always generating sweeps independently of each other, we can selectively group and compute them using an iterative collision solver. Methods for determining optimal group sizes are still being investigated.

2.5 Querying the Simulation

Typical real-time applications using dynamics simulation today synchronize the solver with the main thread [Coumans 2015]. Our approach has the advantage of generating state in a separate thread for main thread to query and display. While not discountable, the querying cost is marginal compared to the cost of stepping the solver. As such, interactive applications simulating advanced dynamics can dedicate a larger budget of their frame time to rendering.

However, if the query mutates the simulation state, the solver runs momentarily on the main thread to update invalidated state. Despite this, invalidated state is usually just a fraction of the total state, so this process will be comparatively cheap in most cases.

Table 1 shows results achieved using our technique to simulate 1000 interactive bodies colliding at varying frequencies. A 2-7x performance gain was observed with prediction enabled vs. disabled.

REFERENCES

- Erwin Coumans. 2015. Bullet 2.83 Physics SDK Manual. <https://pybullet.org/wordpress/index.php/forum-2/>
- Thilo Grundmann. 2009. Using a Physics Engine to Improve Probabilistic Object Localization. In *Autonome Mobile Systeme 2009*, Rüdiger Dillmann, Jürgen Beyerer, Christoph Stiller, J. Marius Zöllner, and Tobias Gindele (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 169–176.
- John D. Owens, Mike Houston, David Luebke, Simon Green, John E. Stone, and James C. Phillips. 2008. GPU Computing. *Proc. IEEE* 96, 5 (2008), 879–899. <https://doi.org/10.1109/JPROC.2008.917757>
- Alvaro Sanchez-Gonzalez, Jonathan Godwin, Tobias Pfaff, Rex Ying, Jure Leskovec, and Peter W. Battaglia. 2020. Learning to Simulate Complex Physics with Graph Networks. *arXiv:2002.09405 [cs.LG]*
- Alvaro Sanchez-Gonzalez, Nicolas Heess, Jost Tobias Springenberg, Josh Merel, Martin Riedmiller, Raia Hadsell, and Peter Battaglia. 2018. Graph Networks as Learnable Physics Engines for Inference and Control. In *Proceedings of the 35th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 80)*, Jennifer Dy and Andreas Krause (Eds.). PMLR, 4470–4479. <http://proceedings.mlr.press/v80/sanchez-gonzalez18a.html>
- Yanchun Shen and Xingyi Sun. 2011. Research and improvement of collision detection based on oriented bounding box in physics engine. In *2011 IEEE 3rd International Conference on Communication Software and Networks*, 73–76. <https://doi.org/10.1109/ICCSN.2011.6014221>
- Jiajun Wu, Ilker Yildirim, Joseph J Lim, Bill Freeman, and Josh Tenenbaum. 2015. Galileo: Perceiving Physical Object Properties by Integrating a Physics Engine with Deep Learning. In *Advances in Neural Information Processing Systems*, C. Cortes, N. Lawrence, D. Lee, M. Sugiyama, and R. Garnett (Eds.), Vol. 28. Curran Associates, Inc.